

Beam-Spline Mappers in the Mok FSI Case

Theory, Current Implementation, Verification, and Stability Evidence

24 July 2026

1 Scope and canonical setup

This note is the single maintained technical record for `FSI_mok_Test_Case`. It explains the current implementation of `BeamSplineMapper` and `BeamSplineMapper.WithRotationalRecovery`, the analytical benchmark design, the conservative inverse maps, and the retained nonlinear FSI evidence. Historical parameter sweeps and pre-fix output are not part of the current evidence set.

The physical model follows the official Kratos Mok validation case. The two-dimensional wall occupies

$$x \in [0.4965, 0.5015] \text{ m}, \quad y \in [0, 0.25] \text{ m},$$

and its beam abstraction is placed on the centroidal line $x = 0.499$ m. The formal linear-thickness scale is $s = 50$. It is not a numerical tuning parameter: structure and fluid load are scaled together. The beam properties are

$$\begin{aligned} A &= 0.25, & I_{22} &= 2.083333333333336 \times 10^{-2}, \\ I_{33} &= 5.208333333333334 \times 10^{-7}, & J &= 2.083333333333334 \times 10^{-6}. \end{aligned}$$

Fluid `REACTION` is multiplied by +50 before transfer, and `swap_sign` remains active. The retained FSI gate uses

$$\Delta t = 0.05, \quad \alpha = 0.03, \quad N_{\text{coupling}} = 60.$$

2 Plain BeamSplineMapper

2.1 Forward map

The plain mapper is a one-dimensional beam-spline construction. A surface node is projected onto the reference beam chain. Axial translation and torsion are interpolated on the local beam coordinate; transverse centerline translations use spline systems whose derivative data are supplied by the beam rotations. A fixed affine augmentation $\{1, \xi\}$ is used, with

$$\xi = \frac{x - X_c}{a} \in [-1, 1],$$

where X_c is the reference-chain midpoint and $a = L/2$. Derivative rows are scaled by $1/a$. This normalization removes dependence on coordinate translation and avoids the dimensional imbalance of an unscaled beam coordinate.

The destination displacement has the generic form

$$u_d = F_{\text{plain}}(q_b).$$

Its section-offset contribution uses the recovered/interpolated section rotation. Because a finite rotation matrix is used in that contribution, F_{plain} is nonlinear in the rotation degrees of freedom even though the spline solves themselves are linear.

2.2 Conservative inverse

The load map is defined by discrete virtual work:

$$\delta u_d = J_{\text{plain}}(q_b) \delta q_b, \quad Q_b = J_{\text{plain}}(q_b)^\top f_d.$$

The bending contribution therefore cannot be distributed only to the two nodes of the projected segment. Its evaluation functional is first accumulated over all destination forces and is then propagated to the whole beam support through the transpose spline solve. The normalized derivative scaling used in the forward map is also used in this adjoint.

Resolved correctness issue. The earlier inverse path used only a two-node force/lever-arm distribution and was not the transpose of the global spline interpolation. It also omitted part of the section offset induced by bending rotations. The repaired map uses the exact forward tangent and its transpose. The current four directional virtual-work errors are

$$9.88 \times 10^{-10}, \quad 4.52 \times 10^{-10}, \quad 1.41 \times 10^{-10}, \quad 2.40 \times 10^{-9}.$$

All pass the 10^{-7} tangent/adjoint acceptance criterion.

3 BeamSplineMapper with rotational recovery

3.1 Small-rotation field

Ahrem, Beckert, and Wendland reconstruct one vector displacement field $s : \mathbb{R}^3 \rightarrow \mathbb{R}^3$. At structural sites X_i , the generalized interpolation conditions are

$$s(X_i) = g_i, \quad \nabla \times s(X_i) = 2h_i.$$

The second equation follows from infinitesimal kinematics,

$$h = \frac{1}{2} \nabla \times u,$$

and the paper explicitly restricts the interpretation to small angles. With three value and three curl functionals per point, the saddle system is

$$\begin{bmatrix} A_{\Phi, X} & P_X \\ P_X^\top & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} g \\ 2h \\ 0 \end{bmatrix}.$$

In `rotation_recovery_mode = small`, the surface displacement is evaluated directly:

$$u_s = s(X_s).$$

For fixed support this map is linear, $u_s = Hq_b$, so the conservative load map is $Q_b = H^\top f_s$.

3.2 Polynomial levels

The public integer setting `polynomial_level` has the following meaning:

- 0 : automatic rank/conditioning fallback,
- 1 : paper full-3D affine space, 12 vector modes,
- 2 : line-basic space, 9 modes,
- 3 : line-enriched space, 12 modes,
- 4 : centered/scaled Legendre high-order space, 30 modes.

Level 4 replaces dimensional monomials by Legendre polynomials in $\xi \in [-1, 1]$ without reducing the polynomial span. Level 1 can be non-unisolvent on a perfectly collinear value/curl support and is

rejected when its functional matrix is rank deficient. Level 0 selects the highest full-rank level below the conditioning limit; it selected level 4 with a polynomial condition estimate of approximately 1.86×10^2 in the analytical beam test.

Legacy string-valued `polynomial_basis` values are translated with a deprecation warning. Supplying both interfaces is an error.

3.3 Kernel, coordinates, and regularization

For the Kratos Gaussian convention

$$\phi(r) = \exp\left[-\frac{1}{2}(r/h)^2\right],$$

the paper radius d in $\exp(-r^2/d^2)$ satisfies $d = \sqrt{2}h$. All recovery coordinates and their value/curl evaluation functionals use the same centered reference scale. The analytic Gaussian Hessian at $r = 0$,

$$\nabla\nabla\phi(0) = -\frac{1}{h^2}I,$$

is retained in the curl-curl self block. The previous generic zero-distance branch incorrectly replaced this nonzero limit by zero.

Regularization is added only to the kernel/interpolation block. The polynomial constraint block remains exactly zero, as required by the saddle formulation. A rank defect in P is never hidden by regularizing this zero block. The production recovery setting is

$$h = 0.50, \quad \epsilon = 10^{-8}, \quad \text{polynomial_level} = 4.$$

3.4 Finite rigid-section hybrid

The selectable `rotation_recovery_mode = finite` is deliberately labelled an Ahrem-rigid-section hybrid, not a finite-rotation theorem from the Ahrem paper. The Ahrem interpolation supplies a smooth displacement field $s : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ and an infinitesimal rotation field

$$\theta(X) = \frac{1}{2}\nabla \times s(X).$$

The hybrid assumption is added afterwards: at an aerodynamic surface point X_s , the cross-section through the projected beam-center point X_c moves as a rigid section whose finite orientation is described by the rotation vector $\theta_c = \theta(X_c)$. Thus

$$r_0 = X_s - X_c, \quad u_c = s(X_c),$$

and the mapped surface displacement is

$$u_s = u_c + [\exp([\theta_c]_{\times}) - I] r_0. \tag{1}$$

Here $[a]_{\times}b = a \times b$. The offset r_0 is a reference offset; it is never formed from current node coordinates. No logarithm of a rotation matrix is used. This avoids the additional branch ambiguity of recovering a rotation vector from $SO(3)$, especially near an angle of π .

Model status. The small-recovery mode evaluates the Ahrem field directly at the surface node, $u_s = s(X_s)$. The finite hybrid instead evaluates the recovered field at the beam-center projection X_c , converts the recovered local curl into a finite section rotation, and applies that finite section motion to the offset r_0 . Therefore the two modes are different physical models: small mode is a general three-dimensional displacement reconstruction, whereas finite mode imposes a rigid cross-section. Finite mode can exactly reproduce a rigid section rotation when the recovered θ_c is the correct rotation vector, but this finite interpretation is an additional beam kinematic assumption and not a consequence of the original small-rotation curl identity.

3.4.1 Forward calculation workflow

For one mapping call, the following data are available:

$$\{X_i, u_i, \theta_i\}_{i=1}^{N_b} \quad \text{on the beam support,} \quad X_s \quad \text{on the surface.}$$

The vectors u_i and θ_i are read from the origin beam **DISPLACEMENT** and **ROTATION** variables. The finite hybrid is computed as follows.

1. Project X_s onto the reference beam chain and obtain the projected center point X_c . If X_s projects onto the segment $(i, i+1)$ with line coordinate $\xi \in [0, 1]$, then

$$X_c = (1 - \xi)X_i + \xi X_{i+1}, \quad r_0 = X_s - X_c.$$

2. Assemble the rotational-recovery Hermite interpolation system on the support nodes. For each beam node there are six scalar functionals: three value functionals $L_{i,a}^u$ and three curl functionals $L_{i,a}^\omega$,

$$L_{i,a}^u(s) = e_a^\top s(X_i), \quad L_{i,a}^\omega(s) = e_a^\top \nabla \times s(X_i), \quad a \in \{x, y, z\}.$$

The right-hand side is

$$b = [u_1, \dots, u_{N_b}, 2\theta_1, \dots, 2\theta_{N_b}]^\top,$$

where the factor 2 comes from $\nabla \times s = 2\theta$.

3. Solve the saddle system

$$\begin{bmatrix} A & P \\ P^\top & 0 \end{bmatrix} \begin{bmatrix} a \\ c \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}. \quad (2)$$

The matrix A is built by applying all value and curl functionals to the vector-valued kernel; P contains the selected vector polynomial modes. Regularization, when used, is added only to the kernel block A , never to the zero constraint block.

4. Evaluate the recovered center displacement and recovered center rotation at X_c :

$$u_c = s(X_c), \quad \theta_c = \frac{1}{2} \nabla \times s(X_c).$$

In matrix form this is

$$u_c = E_u(X_c) \begin{bmatrix} a \\ c \end{bmatrix}, \quad \theta_c = \frac{1}{2} E_\omega(X_c) \begin{bmatrix} a \\ c \end{bmatrix},$$

where E_u evaluates the value of the interpolant and E_ω evaluates its curl.

5. Convert the rotation vector θ_c to a rotation matrix. Let

$$\varphi = \|\theta_c\|, \quad K = [\theta_c]_\times.$$

Rodrigues' formula gives

$$R(\theta_c) = I + \frac{\sin \varphi}{\varphi} K + \frac{1 - \cos \varphi}{\varphi^2} K^2. \quad (3)$$

For $\varphi \approx 0$, the code uses the corresponding Taylor limits, $R \approx I + K + \frac{1}{2} K^2$.

6. Apply the rigid-section offset and write the surface displacement:

$$u_s = u_c + (R(\theta_c) - I) r_0.$$

This workflow is local to one destination node after the global recovery coefficients are known. In practice the recovery coefficients are cached per beam chain/support and reused for all destination nodes that belong to that support.

3.4.2 Conservative inverse workflow

The finite map is nonlinear because $R(\theta_c)$ contains trigonometric functions of the recovered rotation vector. Its conservative inverse is therefore the current-state tangent transpose:

$$Q_b = J(q_b)^\top f_s, \quad J(q_b) = \frac{\partial u_s}{\partial q_b}. \quad (4)$$

Here q_b collects the beam nodal translations and rotations, and Q_b contains the corresponding work-conjugate beam forces and moments. The defining condition is discrete virtual work,

$$f_s^\top \delta u_s = Q_b^\top \delta q_b.$$

For one surface force f_s , variation of (1) gives

$$\delta u_s = \delta u_c + \delta R(\theta_c) r_0. \quad (5)$$

The first term is transferred by the transpose of the value-evaluation operator $E_u(X_c)$. The second term is transferred through the rotation evaluation operator $E_\omega(X_c)$ and the derivative of the exponential map.

For a rotation vector θ , an increment $\delta\theta$ changes the rotated offset according to

$$\delta(R(\theta)r_0) = -R(\theta)[r_0]_\times J_r(\theta) \delta\theta, \quad (6)$$

where J_r is the right Jacobian of $SO(3)$,

$$J_r(\theta) = I - \frac{1 - \cos \varphi}{\varphi^2} [\theta]_\times + \frac{\varphi - \sin \varphi}{\varphi^3} [\theta]_\times^2, \quad \varphi = \|\theta\|. \quad (7)$$

Using $a^\top (B \delta\theta) = (B^\top a)^\top \delta\theta$, the generalized force conjugate to the recovered rotation vector is

$$g_\theta = [-R(\theta_c)[r_0]_\times J_r(\theta_c)]^\top f_s = J_r(\theta_c)^\top [r_0]_\times R(\theta_c)^\top f_s. \quad (8)$$

At zero rotation, $J_r = I$ and $R = I$, so

$$g_\theta = [r_0]_\times f_s = r_0 \times f_s,$$

which is the usual lever-arm moment. At finite rotation, g_θ is not generally equal to this naive moment. It is the component that is work-conjugate to the selected rotation-vector parameterization.

The adjoint accumulation for the recovery coefficients can be written compactly as

$$\delta W = f_s^\top E_u(X_c) \delta z + g_\theta^\top \left[\frac{1}{2} E_\omega(X_c) \delta z \right], \quad z = \begin{bmatrix} a \\ c \end{bmatrix}.$$

Therefore the adjoint right-hand side at the interpolation-coefficient level is

$$r_z = E_u(X_c)^\top f_s + \frac{1}{2} E_\omega(X_c)^\top g_\theta. \quad (9)$$

Solving the transpose of the saddle system,

$$\begin{bmatrix} A & P \\ P^\top & 0 \end{bmatrix}^\top y = r_z,$$

propagates this destination force back to the original value and curl functionals. The entries associated with value data become nodal force contributions; the entries associated with curl data are multiplied by the same factor 2 convention in the forward right-hand side and become work-conjugate nodal moment contributions. This is the algebraic reason why the inverse map must be treated as the exact transpose of the current forward tangent, rather than as an independent lever-arm distribution.

3.4.3 Minimal hand calculation

The following single-section example is sufficient to check the finite rigid-section part by hand after u_c and θ_c have been recovered. Assume the beam center projection is

$$X_c = (0, 0, 0), \quad X_s = (0, 1, 0), \quad r_0 = (0, 1, 0).$$

Suppose the recovery solve gives

$$u_c = (0.10, 0, 0), \quad \theta_c = (0, 0, \alpha), \quad \alpha = \frac{\pi}{2}.$$

Then

$$R(\theta_c) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad R(\theta_c)r_0 = (-1, 0, 0).$$

The finite rigid-section displacement is

$$u_s = (0.10, 0, 0) + [(-1, 0, 0) - (0, 1, 0)] = (-0.90, -1.00, 0).$$

The corresponding small-rotation approximation would instead use $\theta_c \times r_0 = (-\pi/2, 0, 0)$, giving

$$u_s^{small} = (0.10, 0, 0) + (-1.5708, 0, 0) = (-1.4708, 0, 0).$$

This hand calculation shows the modelling difference: finite mode rotates the reference offset as a rigid vector, whereas the small formula keeps only the first-order term.

For the inverse of the same hand case, take a surface force

$$f_s = (0, 1, 0).$$

At $\alpha = \pi/2$, $R^T f_s = (1, 0, 0)$ and $[r_0]_\times R^T f_s = (0, 0, -1)$. Since the rotation is purely about z , the z -component of J_r^T is unchanged, and

$$g_{\theta_z} = -1.$$

Thus a positive y -force at the rotated point performs negative virtual work against a positive additional z -rotation in this configuration. Together with the direct translational contribution f_s , this is the quantity that must be propagated back through the transpose recovery system. The sign follows from differentiating the finite rotated offset, not from an ad-hoc moment rule.

4 Common geometric and algebraic corrections

The following defects were resolved in both mapper implementations.

1. Mapper search no longer writes reference coordinates into the beam nodes' current coordinates. Projection may use reference geometry, but it has no observable side effect on the structural state.
2. When an approximate projection coordinate is clamped, its shape functions, projection point, distance, and section offset are rebuilt from that same clamped coordinate.
3. The local frame is constructed from the Cartesian axis least aligned with the beam tangent, followed by orthogonal projection and a right-handed cross product. Negative axes, near-axis directions, and nearly collinear segments no longer enter discontinuous equality branches.
4. Reference-chain midpoint and half-length scaling are used consistently in values, derivatives, curls, moment functionals, and inverse functionals.
5. Explicit inverse matrices are not stored. Forward systems are solved, and inverse functionals are aggregated per beam chain before solving $A^T z = b$.
6. `swap_sign` is applied only at the physical load accumulation; it is not mixed into the mathematical transpose.

5 Analytical benchmark design

Mapper accuracy is measured only against prescribed analytical fields, never against another mapper. For surface-node values,

$$e_{\text{relL2}} = \sqrt{\frac{\sum_i \|u_i^{\text{map}} - u_i^{\text{exact}}\|^2}{\sum_i \|u_i^{\text{exact}}\|^2}}.$$

Exact-span tests with zero regularization use a practical solver-level target of 10^{-10} , relaxed to 10^{-9} when the assembled saddle solve and coordinate transforms make 10^{-10} marginal. Fields that centerline values and curl cannot uniquely identify are reported as *non-identifiable diagnostics*, not exact-reproduction failures.

5.1 Forward verification

The core suite contains rigid translation and infinitesimal rotation, axial motion, bending, torsion, curl-compatible fields, polynomial-level reproduction, nonuniform spacing, translated/scaled coordinates, negative axes, and near-collinear geometry. Finite-mode checks prescribe exact rigid rotations of $1^\circ, 30^\circ, 90^\circ, 170^\circ$. Every test also checks that mapper construction leaves current coordinates unchanged.

The current rigid-rotation results are:

	1°	30°	90°	170°
BeamSplineMapper	5.81×10^{-6}	5.22×10^{-3}	4.51×10^{-2}	8.83×10^{-2}
Recovery small	9.18×10^{-4}	2.89×10^{-2}	1.13×10^{-1}	3.13×10^{-1}
Recovery finite	3.98×10^{-15}	4.47×10^{-16}	5.71×10^{-16}	1.18×10^{-15}

These results quantify kinematic compatibility; they do not establish FSI stability.

5.2 Backward/adjoint verification

For a prescribed state q , perturbation p , and surface force f , the centered finite-difference directional derivative is compared with the inverse-map work:

$$\frac{f^\top [F(q + \varepsilon p) - F(q - \varepsilon p)]}{2\varepsilon} \quad \text{versus} \quad p^\top J(q)^\top f.$$

The production Jacobian is analytical; finite differences are used only for verification. Bending about both transverse axes, torsion, and a mixed field are included. All current plain, recovery-small, and recovery-finite cases pass. Their worst relative discrepancies are respectively

$$2.40 \times 10^{-9}, \quad 2.24 \times 10^{-9}, \quad 2.57 \times 10^{-9}.$$

6 FSI benchmark design

The FSI runner supports NearestNeighbor, BeamMapper_CoRotation, BeamSplineMapper, and both recovery modes, together with scale, time step, relaxation, coupling-iteration limit, kernel radius, regularization, polynomial level, structural strategy, and target end time. Each trial:

1. snapshots every modified JSON, MDPA, and material file;
2. synchronizes section and fluid-load scaling;
3. writes point history, CFD/CSM VTK, full log, settings, conclusion, and summary JSON into a unique directory;
4. monitors nonfinite output, coupling failure, and repeated structural nonconvergence;

5. stops invalid runs and verifies restoration by file hashes.

The stability gate is $0.5 \rightarrow 4.5 \rightarrow 10 \rightarrow 25$ s. Failure of one gate prevents an unsupported longer run. The analytical suite remains the correctness reference. In addition, the FSI runner uses a fixed, physically scaled NearestNeighbor point-A u_x history as a trajectory reference:

$$e_{\text{NN}} = \sqrt{\frac{\sum_{t \in \mathcal{T}_{\text{valid}}} (u_{\text{mapper}}(t) - u_{\text{NN}}(t))^2}{\sum_{t \in \mathcal{T}_{\text{valid}}} u_{\text{NN}}(t)^2}}.$$

Here $\mathcal{T}_{\text{valid}}$ is the exact intersection of finite sample times, truncated at the mapper’s last accepted time; no interpolation or nonconverged output is used. This metric compares nonlinear FSI trajectories, but does not replace the prescribed-field analytical tests.

7 Retained post-fix FSI results

All retained trials use $s = 50$, $\Delta t = 0.05$, $\alpha = 0.03$, 60 coupling iterations, synchronized scaling, and `swap_sign`.

Mapper/mode	Target	Last valid	e_{NN}	Failure
NearestNeighbor reference	4.5 s	4.50 s	0	completed
BeamSplineMapper, normalized	4.5 s	3.05 s	1.0315×10^{-2}	structural nonconvergence, step 62 at 3.10 s
Recovery, level 4, small	4.5 s	4.20 s	3.8945×10^{-2}	structural nonconvergence, step 85 at 4.25 s
Recovery, level 4, finite	4.5 s	3.05 s	1.0242×10^{-2}	structural nonconvergence, step 62 at 3.10 s

The error column uses each case’s own valid interval. Since the small-recovery case runs longer, a fair all-beam comparison also uses the shared 0.05–3.05 s interval (61 samples): plain, recovery-small, and recovery-finite respectively give

$$1.0315 \times 10^{-2}, \quad 1.7555 \times 10^{-2}, \quad 1.0242 \times 10^{-2}.$$

Thus the small-mode value of 3.8945×10^{-2} over its longer 0.05–4.20 s interval must not be directly ranked against the two shorter values.

All four trial summaries report successful restoration. The NN reference completes 4.5 s, but no beam mapper reaches the 4.5 s gate, so no post-fix beam run was promoted to 10 or 25 s. The finite recovery and normalized plain histories are almost identical at their limit: their point-A u_x values at $t = 3.05$ s are 0.01611986 and 0.01612157 m. Recovery-small reproduces the earlier step-85 limit after coordinate normalization and the Legendre polynomial correction. Therefore the old high-order monomial conditioning was not the root cause of that event.

The analytical checks exclude coordinate mutation, a discontinuous projection definition, missing Gaussian self-Hessian, a non-adjoint load map, and an incorrect finite $\text{SO}(3)$ tangent as current explanations. FSI fields remain finite up to the last accepted step, after which the structural nonlinear solve repeatedly fails. The evidence presently classifies the terminal event primarily as a structural/coupled-trajectory nonconvergence, not as a mapper singularity or mapper-produced NaN. A structure-only replay of steps 62 and 85 is the next mechanism-isolating experiment.

8 Reproducibility and retained records

The canonical scripts are under `Scripts/`:

- `run_fsi_benchmark.py`: parameterized, restoring FSI runner;
- `validate_beam_spline_forward.py` and `validate_beam_spline_adjoint.py`;
- `validate_recovery_forward.py` and `validate_recovery_adjoint.py`;
- the conditioning, Newton-history, VTK-history, and load-consistency analyzers.

Current analytical results are in `TestCase_Output/MapperVerification`. Each validation script defines its canonical output directory near the top of the file and, on rerun, replaces only that exact direct child of `MapperVerification`. This keeps the normal invocation reproducible without accumulating versioned copies. Current nonlinear FSI evidence is restricted to the NN reference and the three beam $t = 4.5$ gate directories under `TestCase_Output/StabilityTrials_s50`. The FSI runner avoids aliasing NN point-A and point-B output streams and writes both own-valid-range and common-range comparisons to `nn_rel_l2_comparison.json` and `nn_rel_l2_comparison.csv`. Every numerical conclusion above can be reconstructed from these summaries and the individual logs.

9 Remaining theoretical decisions

1. Confirm whether the desired recovery mapper is the paper-faithful, general three-dimensional Ahrem displacement field or a rigid-section finite-rotation beam surface model. They are different nonlinear FSI models.
2. Confirm the precise semantics of the structural element's `ROTATION`: total exponential-coordinate vector, incremental rotation, or another composed parameter. The finite hybrid requires a compatible total-state interpretation.
3. Confirm whether rigid cross-sections are a desired physical constraint. If section warping is relevant, the finite hybrid is too restrictive.
4. If a fully geometrically exact recovery is required, consider separate interpolation of translation and orientation on $SO(3)$, with a consistent tangent transpose, rather than assigning finite physical meaning to $\frac{1}{2}\nabla \times s$.